DESAFIO TRAINEE TECH SPACECOM 2027
PROCESSO SELETIVO — PRIMEIRA ETAPA TÉCNICA

EMERSON

DOCUMENTAÇÃO TÉCNICA: SISTEMA DE DETECÇÃO DE ENCONTROS EM GEOLOCALIZAÇÃO

Curitiba
2026

EMERSON

DOCUMENTAÇÃO TÉCNICA: SISTEMA DE DETECÇÃO DE ENCONTROS EM GEOLOCALIZAÇÃO

Documentação técnica apresentada como parte da primeira etapa do processo seletivo do Programa Trainee Tech Spacecom 2027, referente ao desafio de detecção de encontros por geolocalização.

Curitiba
2026

RESUMO

Este documento descreve a arquitetura e as decisões técnicas de um sistema desenvolvido em TypeScript/Node.js para detectar encontros físicos entre indivíduos monitorados por geolocalização (tornozeleiras eletrônicas e rastreadores de vítimas com medida protetiva), no contexto do Desafio Trainee Tech Spacecom 2027. O sistema processa arquivos de posicionamento separados por ponto e vírgula, calcula distâncias pela fórmula de Haversine e classifica encontros em três categorias (genérico, de facção e de medida protetiva), respeitando um raio de 20 metros. São documentados, módulo a módulo, o funcionamento de cada função, a fundamentação teórica de cada técnica empregada e a justificativa de organização do código. Também são apresentadas evidências empíricas de desempenho obtidas em testes com os datasets oficiais do desafio.

Palavras-chave: Geolocalização. Fórmula de Haversine. TypeScript. Streaming de dados. Indexação espacial.

SUMÁRIO

1 INTRODUÇÃO
2 ARQUITETURA GERAL DO SISTEMA
3 MODELAGEM DE DADOS (types.ts)
4 CÁLCULO DE DISTÂNCIA GEOGRÁFICA (haversine.ts)
5 LEITURA E PROCESSAMENTO EM STREAMING (parser.ts)
6 DETECÇÃO DE ENCONTROS VIA INDEXAÇÃO ESPACIAL EM GRADE (detector.ts)
7 ORQUESTRAÇÃO, VALIDAÇÃO E MEDIÇÃO DE DESEMPENHO (main.ts)
8 CONSIDERAÇÕES DE DESEMPENHO E ESCALABILIDADE
9 CONFORMIDADE COM OS REQUISITOS DO DESAFIO
10 CONSIDERAÇÕES FINAIS
REFERÊNCIAS

1 INTRODUÇÃO

O Desafio Trainee Tech Spacecom 2027 propõe o desenvolvimento de um algoritmo capaz de processar grandes volumes de dados de geolocalização e identificar, para cada instante de tempo registrado, pares de indivíduos monitorados que estejam fisicamente próximos (distância menor que 20 metros), classificando esse encontro conforme a relação entre os envolvidos. O enunciado exige o uso de JavaScript ou TypeScript em ambiente Node.js, o uso da fórmula de Haversine com raio terrestre de 6.371.000 metros, e estabelece como critério de desempate a eficiência da solução.

Este documento tem como objetivo registrar, de forma técnica e rastreável a fontes externas, como cada parte do sistema funciona e por que foi organizada da maneira como está, tanto para fins de defesa da solução perante a banca avaliadora quanto como material de referência para manutenção futura do código.

2 ARQUITETURA GERAL DO SISTEMA

O sistema é dividido em cinco módulos TypeScript, cada um com uma única responsabilidade, seguindo o princípio de separação de responsabilidades amplamente adotado em engenharia de software:

Módulo	**Responsabilidade**
types.ts	Definição dos contratos de dados (interfaces)
haversine.ts	Cálculo de distância geográfica entre dois pontos
parser.ts	Leitura do arquivo de entrada e agrupamento por timestamp, em streaming
detector.ts	Detecção e classificação dos encontros dentro de cada timestamp
main.ts	Orquestração do fluxo, medição de tempo e validação contra o gabarito

O fluxo de dados é estritamente sequencial e unidirecional: parser.ts entrega um grupo de posições de um único timestamp por vez para detector.ts, que devolve os encontros encontrados naquele grupo para main.ts, que acumula o resultado e grava o JSON de saída. Essa escolha de arquitetura — processar e descartar cada timestamp imediatamente, em vez de carregar o arquivo inteiro antes de processar — foi uma correção motivada por um erro de estouro de memória (heap out of memory) observado durante os testes com os datasets oficiais de maior volume (423 MB e 638 MB), e está detalhada na seção 5.

3 MODELAGEM DE DADOS (TYPES.TS)

O módulo types.ts define três estruturas centrais: Posicionamento (uma linha do arquivo de entrada já validada e tipada), TipoEncontro (um tipo literal restrito aos três valores aceitos pelo enunciado: “generico”, “faccao”, “protetiva”) e Encontro (a estrutura de saída). O uso de tipos explícitos do TypeScript, em vez de JavaScript puro, segue a recomendação da própria documentação oficial da linguagem para sistemas em que o contrato de dados precisa ser verificado em tempo de compilação, reduzindo a chance de um campo ser lido ou escrito com o nome ou tipo errado (MICROSOFT, [s.d.]). Essa garantia é relevante neste desafio porque a fórmula de pontuação (score = soma_ec − soma_ei) penaliza um falso positivo tanto quanto recompensa um acerto — um erro silencioso de tipagem teria custo direto na nota.

idsRestricaoMedida é modelado como Set<number>, e não como array, porque a única operação necessária sobre essa coleção é a verificação de pertencimento (“o id X está na lista de agressores restritos?”), que em um Set tem complexidade média O(1), contra O(n) em um array (MOZILLA, [s.d.]a).

4 CÁLCULO DE DISTÂNCIA GEOGRÁFICA (HAVERSINE.TS)

O enunciado exige explicitamente o uso da fórmula de Haversine para o cálculo da distância entre dois pontos de latitude/longitude, considerando o raio da Terra como 6.371.000 metros. A fórmula implementada segue exatamente a formulação clássica:

```
a = sin²(Δφ/2) + cos(φ1) · cos(φ2) · sin²(Δλ/2)
c = 2 · atan2(√a, √(1−a))
d = R · c
```

onde φ representa a latitude, λ a longitude e R o raio terrestre. Essa é a formulação de referência amplamente documentada por Veness (VENESS, [s.d.]) e descrita formalmente no verbete de distância ortodrômica da Wikipédia (WIKIPEDIA, [s.d.]a), que também observa que a fórmula de Haversine permanece numericamente bem-condicionada mesmo para distâncias pequenas — como as de até 20 metros tratadas neste desafio — ao contrário de alternativas baseadas na lei dos cossenos esférica, que perdem precisão nessa faixa.

A implementação foi validada diretamente contra o gabarito oficial do desafio (respostas1.json): os valores de distanciaMetros calculados pelo sistema bateram exatamente, em todas as casas decimais, com os valores fornecidos pela organização do desafio para os pares de exemplo, o que confirma a correção da implementação.

5 LEITURA E PROCESSAMENTO EM STREAMING (PARSER.TS)

5.1 Problema original

A primeira versão do sistema lia o arquivo inteiro e agrupava todas as posições em um único Map em memória antes de iniciar a detecção. Essa abordagem funcionou nos datasets menores (1 a 5), mas causou um erro fatal de JavaScript heap out of memory ao processar os datasets 6 e 7 (423 MB e 638 MB, respectivamente), pois o volume total de posições parseadas excedeu o limite padrão de heap do V8.

5.2 Solução adotada

A correção substituiu esse carregamento integral por um async generator (gruposDoDataset), um recurso nativo do ECMAScript que permite produzir valores de forma assíncrona e sob demanda, consumidos com a sintaxe for await...of (MOZILLA, [s.d.]b; MOZILLA, [s.d.]c). A leitura do arquivo em si é feita linha a linha através do módulo readline do Node.js, aplicado sobre um stream de leitura (fs.createReadStream), conforme documentado na API oficial do Node.js (NODE.JS, [s.d.]a; NODE.JS, [s.d.]b), um padrão recomendado pela própria documentação para o caso de uso de “parsing linha a linha de um arquivo”.

O gerador acumula as posições de um timestamp em um buffer e, assim que detecta uma linha pertencente a um timestamp diferente, entrega (yield) o buffer completo ao chamador, que o processa e descarta antes da leitura continuar. Esse desenho pressupõe que o arquivo venha agrupado por timestamp, padrão observado em todos os datasets de exemplo e oficiais, e mantém o uso de memória proporcional ao número de monitorados em um único timestamp, nunca ao tamanho do arquivo inteiro.

5.3 Validação empírica

A eficácia da correção foi comprovada tanto nos sete datasets oficiais (que passaram a processar sem erro de memória) quanto em um teste sintético de estresse com 900 mil linhas (3.000 monitorados × 300 timestamps, incluindo um agrupamento artificialmente denso de pontos): o processamento completo consumiu aproximadamente 39 MB de heap, contra um estouro de mais de 4 GB na versão anterior.

6 DETECÇÃO DE ENCONTROS VIA INDEXAÇÃO ESPACIAL EM GRADE (DETECTOR.TS)

6.1 O problema da comparação par a par

Para cada timestamp, o problema central é: dado um conjunto de k posições simultâneas, quais pares estão a menos de 20 metros um do outro? A abordagem ingênua, comparar cada posição com todas as demais, tem complexidade $O(k^2)$, que se torna impraticável quando k cresce (ZHENG et al., 2006). Esse crescimento quadrático foi observado empiricamente: o tempo de processamento do dataset 6 chegou a quase 20 segundos em uma versão intermediária da solução, evidenciando a necessidade de uma estrutura de indexação espacial.

6.2 Solução: grade espacial (uniform grid)

A solução final organiza, para cada timestamp, os monitorados em uma grade espacial 2D, técnica amplamente descrita na literatura de bancos de dados espaciais e de detecção de colisão como uma forma eficiente de restringir comparações de distância a vizinhanças locais (ZHENG et al., 2006). Cada célula da grade tem um lado aproximadamente igual ao limite de 20 metros do desafio. Para cada monitorado, apenas os monitorados presentes na própria célula e nas oito células vizinhas (vizinhança 3×3) são considerados candidatos a um encontro, cobertura suficiente para garantir que nenhum par realmente a menos de 20 metros seja perdido, já que a distância máxima coberta pela vizinhança sempre excede o raio de busca.

Por se tratar de dados sobre a superfície terrestre, o tamanho de uma célula em graus de longitude varia com a latitude (1° de longitude corresponde a menos metros quanto mais próximo dos polos), enquanto o tamanho em graus de latitude é praticamente constante em qualquer ponto do planeta. Por isso, a dimensão de latitude da célula usa uma constante fixa (≈111.320 m por grau), e a dimensão de longitude é recalculada a partir da latitude de cada ponto individualmente. Essa abordagem reduz a complexidade média de $O(k^2)$ para $O(k)$ por timestamp (ZHENG et al., 2006), independentemente do número total de monitorados, desde que eles não estejam todos fisicamente concentrados na mesma célula de 20 metros, cenário que corresponderia, de qualquer forma, a um encontro real e, portanto, inevitável de se comparar.

6.3 Classificação do tipo de encontro

A função classificarEncontro aplica uma ordem de prioridade — protetiva > facção > genérico — quando um par de monitorados atenderia a mais de um critério simultaneamente:

a) Protetiva: um dos dois tem medidaProtetiva = true e o identificador do outro está presente em seu idsRestricaoMedida (verificado nos dois sentidos da relação);
b) Facção: ambos têm faccaoMonitorado preenchido e idêntico;
c) Genérico: qualquer outro par que se encontre fisicamente.

Essa prioridade não está explicitada literalmente no enunciado do desafio, sendo a principal decisão de negócio tomada de forma independente pela equipe de desenvolvimento: um encontro de medida protetiva representa um risco mais grave e específico do que um encontro genérico entre dois monitorados quaisquer, justificando sua precedência na classificação.

6.4 Garantia de unicidade

O enunciado exige que cada par seja registrado uma única vez por timestamp. Essa garantia é obtida estruturalmente: dentro de cada timestamp, um par (p, q) só é avaliado quando o identificador de q é maior que o de p, tornando estruturalmente impossível que o mesmo par seja avaliado, e portanto registrado, duas vezes.

7 ORQUESTRAÇÃO, VALIDAÇÃO E MEDIÇÃO DE DESEMPENHO (MAIN.TS)

O módulo main.ts varre a pasta data/ em busca de arquivos datasetN.txt, processa cada um através do gerador de parser.ts e do detector de detector.ts, grava o resultado em output/resultado-datasetN.json e mede o tempo de execução de cada dataset com process.hrtime.bigint(), API de alta resolução do próprio Node.js para medição de intervalos de tempo. Quando um arquivo respostasN.json correspondente está presente (gabarito dos datasets de teste 1 e 2), o sistema compara automaticamente o resultado obtido com o gabarito e reproduz localmente a fórmula de pontuação do desafio (score = soma_ec − soma_ei), permitindo validar a lógica antes do envio da solução.

8 CONSIDERAÇÕES DE DESEMPENHO E ESCALABILIDADE

Os tempos de execução medidos nos sete datasets oficiais, após as correções descritas nas seções 5 e 6, foram:

Dataset	Tamanho aproximado	Tempo de execução
1	261 KB	23,6 ms
2	1,3 MB	94,2 ms
3	5,3 MB	271,1 ms
4	21,1 MB	1,04 s

5	52,8 MB	2,40 s
6	423,4 MB	19,06 s
7	638,2 MB	32,18 s

O crescimento do tempo de execução acompanha o crescimento do volume de dados de forma consistente com uma complexidade próxima de linear em relação ao número de linhas do arquivo, o que é o comportamento esperado de um pipeline de leitura em streaming (complexidade O(n) na leitura, seção 5) combinado a uma detecção de complexidade média O(k) por timestamp (seção 6), em contraste com o comportamento quadrático característico de uma solução por força bruta, cujo crescimento seria desproporcionalmente mais acentuado à medida que o número de monitorados por timestamp aumenta (definição geral de notação assintótica em WIKIPEDIA, [s.d.]b).

9 CONFORMIDADE COM OS REQUISITOS DO DESAFIO

Requisito do enunciado	**Como é atendido**
Tecnologia: JavaScript ou TypeScript (Node.js)	TypeScript compilado para Node.js via tsc, sem frameworks externos
Distância menor que 20 metros	Comparação estrita distancia < 20, nunca <=
Fórmula de Haversine, raio 6.371.000 m	Implementada conforme seção 4, validada contra o gabarito oficial
Comparar apenas o mesmo timestamp exato	Agrupamento por igualdade de string do timestamp, sem parsing de data
Três tipos de encontro	Implementados conforme prioridade da seção 6.3
Unicidade por par e timestamp	Garantida estruturalmente (seção 6.4)
Formato de input e output	Implementados conforme especificação literal do enunciado
Eficiência (critério de desempate)	Streaming + grade espacial (seções 5 e 6); tempos medidos na seção 8

10 CONSIDERAÇÕES FINAIS

O sistema desenvolvido atende aos requisitos funcionais e de formato especificados no enunciado do Desafio Trainee Tech Spacecom 2027, com validação empírica da fórmula de distância contra o gabarito oficial e comprovação prática de escalabilidade nos sete datasets fornecidos, incluindo os dois arquivos de maior volume (423 MB e 638 MB), que inicialmente causavam falha de memória. As duas principais decisões de engenharia, processamento em streaming e indexação espacial em grade, foram adotadas não apenas por fundamentação teórica prévia, mas como resposta direta a um problema de desempenho observado empiricamente durante o desenvolvimento, e tiveram sua eficácia comprovada por meio de testes com dados sintéticos e reais.

REFERÊNCIAS

MICROSOFT. *TypeScript Handbook*. [S.l.: s.n.], [s.d.]. Disponível em: https://www.typescriptlang.org/docs/handbook/intro.html. Acesso em: 30 out. 2026.

MOZILLA. *Map*. MDN Web Docs. [S.l.: s.n.], [s.d.]a. Disponível em: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map. Acesso em: 30 out. 2026.

MOZILLA. *async function**. MDN Web Docs. [S.l.: s.n.], [s.d.]b. Disponível em: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function*. Acesso em: 30 out. 2026.

MOZILLA. *for await...of*. MDN Web Docs. [S.l.: s.n.], [s.d.]c. Disponível em: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for-await...of. Acesso em: 30 out. 2026.

NODE.JS FOUNDATION. *Readline*. Node.js Documentation. [S.l.: s.n.], [s.d.]a. Disponível em: https://nodejs.org/api/readline.html. Acesso em: 30 out. 2026.

NODE.JS FOUNDATION. *File system*. Node.js Documentation. [S.l.: s.n.], [s.d.]b. Disponível em: https://nodejs.org/api/fs.html. Acesso em: 30 out. 2026.

VENESS, Chris. *Calculate distance, bearing and more between latitude/longitude points*. Movable Type Scripts. [S.l.: s.n.], [s.d.]. Disponível em: https://www.movable-type.co.uk/scripts/latlong.html. Acesso em: 30 out. 2026.

WIKIPEDIA. *Great-circle distance*. Wikipedia, the Free Encyclopedia. [S.l.: s.n.], [s.d.]a. Disponível em: https://en.wikipedia.org/wiki/Great-circle_distance. Acesso em: 30 out. 2026.

WIKIPEDIA. *Big O notation*. Wikipedia, the Free Encyclopedia. [S.l.: s.n.], [s.d.]b. Disponível em: https://en.wikipedia.org/wiki/Big_O_notation. Acesso em: 30 out. 2026.

ZHENG, Baihua; XU, Jianliang; LEE, Wang-Chien; LEE, Dik Lun. *Grid-partition index*: a hybrid method for nearest-neighbor queries in wireless location-based services. The VLDB Journal, v. 15, n. 1, p. 21-39, 2006. Disponível em: https://www.comp.hkbu.edu.hk/~xujl/Papers/vldbj06.pdf. Acesso em: 30 out. 2026.